

LEARNING BUG SIGNATURES FROM EXECUTION TRACE DIVERGENCE: PROMISE AND PITFALLS OF DIVERGENCE-AWARE PRE-TRAINING

Anonymous authors

Paper under double-blind review

ABSTRACT

We investigate whether neural models can learn debugging intuition by training on execution trace divergences between correct and buggy program variants. Our approach, Divergence-Aware Pre-training for Repair (DAPR), generates synthetic bugs through controlled mutations, extracts paired execution traces, and trains models to predict mutation types from divergence patterns. While DAPR achieves high accuracy (96.7%) on mutation classification with strong cross-dataset generalization, our extensive ablation studies reveal surprising findings: the success stems primarily from carefully engineered per-step trace features rather than learned debugging intuition, and proposed architectural enhancements like attention mechanisms actually hurt performance. These results highlight a critical gap between learning to classify synthetic mutations and developing transferable debugging capabilities.

1 INTRODUCTION

Neural program repair has made significant progress by treating bug fixing as text-to-text translation (Jiang et al., 2021; Xia & Zhang, 2022). However, these approaches lack grounding in execution semantics—they learn statistical patterns without understanding *why* certain changes fix bugs. We hypothesized that models could develop stronger debugging capabilities by learning from execution trace divergences: the points where buggy program behavior deviates from correct behavior.

This paper presents DAPR (Divergence-Aware Pre-training for Repair), a framework that trains models to classify bug types by analyzing how execution traces diverge between correct programs and their synthetically mutated variants. Our approach generates paired traces, extracts features capturing where and how variable states first differ, and trains a classifier to predict the underlying mutation type.

Our experiments reveal a nuanced picture. DAPR achieves 96.7% accuracy on mutation type classification and generalizes well across different code distributions. However, ablation studies expose critical limitations: (1) performance depends almost entirely on hand-crafted per-step trace features, dropping to 52.7% without them; (2) attention mechanisms actually *decrease* performance; and (3) the model struggles with “wrong variable” mutations, confusing them with off-by-one errors. These findings suggest that while execution traces contain discriminative information, the model learns surface-level correlations rather than deep debugging intuition.

2 RELATED WORK

Neural Program Repair. Learning-based approaches have evolved from neural machine translation (Jiang et al., 2021) to pre-trained code models (Wang et al., 2021b; Feng et al., 2020). AlphaRepair (Xia & Zhang, 2022) demonstrates zero-shot repair, while LLMs show promise on QuixBugs (Prenner & Robbes, 2021; Lin et al., 2017). These treat repair as sequence transduction without explicit execution reasoning.

Execution-Guided Learning. PPOCoder (Shojaee et al., 2023) and RLEF (Gehring et al., 2024) use RL with execution-based rewards, while InterCode (Yang et al., 2023) frames code generation

Table 1: Cross-dataset generalization results. DAPR achieves consistently high accuracy across all test distributions, demonstrating robust generalization from synthetic training data.

	Synthetic	CodeXGLUE	MBPP	HumanEval
Test Accuracy (%)	96.7	96.0	98.0	96.6

as interactive RL. However, these use binary pass/fail signals, discarding rich information about *how* execution diverges. TraceFixer (Bouzenia et al., 2023) uses traces as input but requires user-provided correct states. TraceDiff (Suzuki et al., 2017) uses trace comparison for visualization rather than learning.

Contrastive Pre-training for Code. Contrastive learning has proven effective for code (Wang et al., 2021a; 2022; Du & Yu, 2023), contrasting different views (AST, data flow, text) but not incorporating execution semantics.

3 METHOD

Synthetic Bug Generation. Following mutation testing research (Tufano et al., 2018), we generate bugs by applying controlled mutations: *off-by-one* (loop bound errors), *wrong operator* (arithmetic/comparison substitution), *wrong variable* (identifier confusion), and *boundary error* (initialization issues).

Trace Extraction and Features. For each correct-buggy pair, we instrument both versions to record variable states. We extract a 64-dimensional feature vector: (1) *global features* (4 dim): normalized divergence index, relative trace length difference; (2) *per-step features* (60 dim): for each of 20 trace positions, the value difference, ratio, and divergence indicator.

Model Architecture. Our classifier uses a two-layer feedforward encoder with LayerNorm and dropout (0.2), followed by a classification head. We also experimented with attention and auxiliary divergence prediction, though these did not improve performance.

4 RESULTS AND ANALYSIS

Setup. We generate 2,000 synthetic program pairs (500 per mutation type), split 70/15/15. We train for up to 80 epochs with early stopping (patience=15), using AdamW (lr=0.002, weight decay=0.01).

Main Results. DAPR achieves 96.7% test accuracy with strong cross-dataset generalization (Table 1). Training dynamics reveal an unusual pattern: validation loss consistently stays below training loss, potentially indicating that the validation set is easier or that strong regularization (dropout, weight decay) affects training more than validation.

Error Analysis. The confusion matrix (Figure 1) reveals that errors concentrate exclusively on “wrong variable” mutations, while the other three classes achieve perfect classification. This systematic pattern suggests that variable swaps can produce divergence signatures similar to arithmetic errors—when a swapped variable affects loop indices or accumulator values, the resulting trace divergence mimics off-by-one or operator errors.

Critical Ablation Findings. Our ablation study (Figure 2) exposes the true source of DAPR’s performance. Removing per-step trace features causes accuracy to collapse from 96.7% to 52.7%—only marginally above random chance (25%). This demonstrates that success depends entirely on hand-crafted features encoding *how* values differ at each trace position, rather than learned abstract patterns from global divergence information.

Most surprisingly, removing the attention mechanism *improves* accuracy to 97.7%. We hypothesized attention would help the model focus on informative trace positions, but for fixed-length feature vectors with explicit divergence indicators, attention adds unnecessary complexity and training instability. Similarly, the auxiliary divergence location prediction task saturates to 100% accuracy

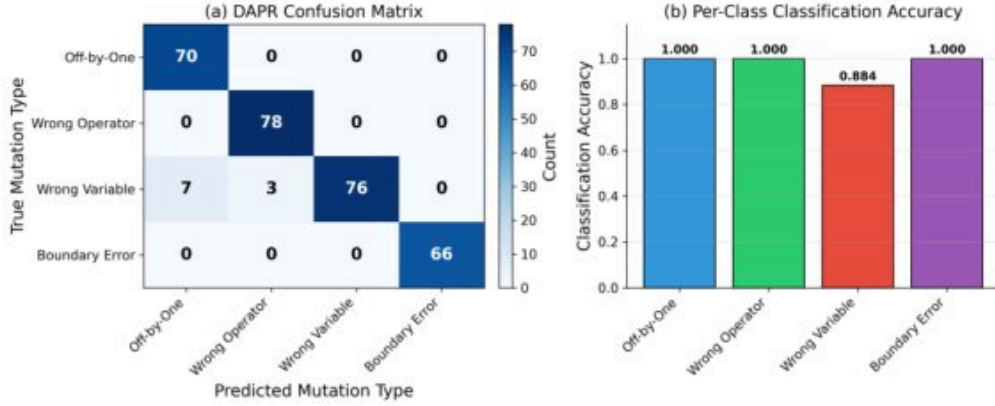


Figure 1: Error analysis. (a) Confusion matrix reveals systematic errors: “wrong variable” mutations are confused with “off-by-one” (7 cases) and “wrong operator” (3 cases). (b) Per-class accuracy shows perfect classification for three mutation types but only 88.4% for “wrong variable.”

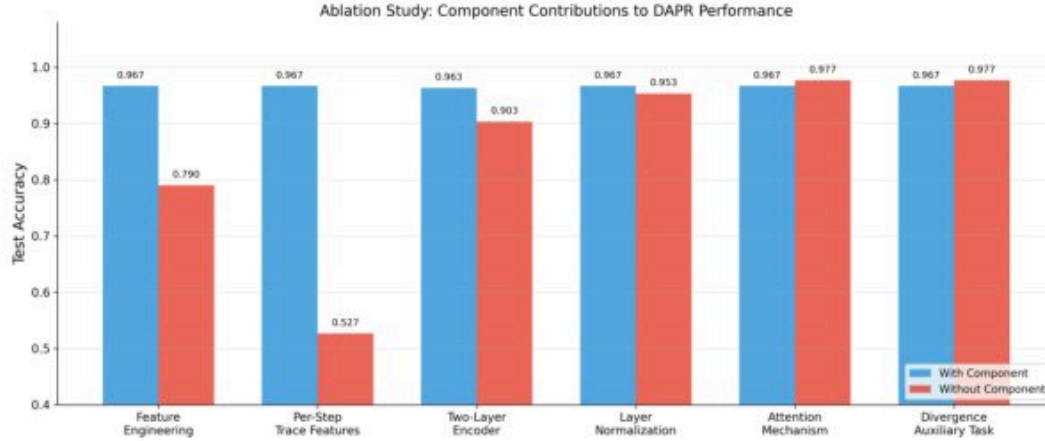


Figure 2: Ablation study revealing critical component contributions. Per-step trace features are essential—removing them causes accuracy to collapse from 96.7% to 52.7%, barely above the 25% random baseline. Counter-intuitively, removing attention or the divergence auxiliary task *improves* performance to 97.7%.

within 2–3 epochs, indicating it is trivially solvable and provides no gradient signal to improve the main classification objective.

5 CONCLUSION

We investigated whether models can learn debugging intuition from execution trace divergences. While DAPR achieves high accuracy on synthetic mutation classification, ablations reveal success stems from engineered features rather than learned capabilities. The attention mechanism and auxiliary task—designed to help reason about trace structure—actually hurt performance. These findings highlight a fundamental challenge: learning to classify synthetic mutations based on hand-crafted features does not transfer to understanding *why* programs fail. Future work should explore richer trace representations (e.g., learned embeddings of execution states) and evaluation on real bugs from benchmarks like Defects4J (Just et al., 2014).

REFERENCES

- Islem Bouzenia, Yangruibo Ding, Kexin Pei, Baishakhi Ray, and Michael Pradel. Tracefixer: Execution trace-driven program repair. 2023.
- Yali Du and Zhongxing Yu. Pre-training code representation with semantic flow graph for effective bug localization. *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023.
- Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. Codebert: A pre-trained model for programming and natural languages. *ArXiv*, abs/2002.08155, 2020.
- Jonas Gehring, Kunhao Zheng, Jade Copet, Vegard Mella, Taco Cohen, and Gabriel Synnaeve. Rlef: Grounding code llms in execution feedback with reinforcement learning. *ArXiv*, abs/2410.02089, 2024.
- Nan Jiang, Thibaud Lutellier, and Lin Tan. *CURE: Code-Aware Neural Machine Translation for Automatic Program Repair*. 2021.
- René Just, D. Jalali, and Michael D. Ernst. Defects4j: a database of existing faults to enable controlled testing studies for java programs. pp. 437–440, 2014.
- Derrick Lin, James Koppel, Angela Chen, and Armando Solar-Lezama. Quixbugs: a multi-lingual program repair benchmark set based on the quixey challenge. *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity*, 2017.
- Julian Aron Prenner and R. Robbes. Automatic program repair with openai’s codex: Evaluating quixbugs. *ArXiv*, abs/2111.03922, 2021.
- P. Shojaei, Aneesh Jain, Sindhu Tipirneni, and Chandan K. Reddy. Execution-based code generation using deep reinforcement learning. *ArXiv*, abs/2301.13816, 2023.
- Ryo Suzuki, Gustavo Soares, Andrew Head, Elena L. Glassman, Ruan Reis, Melina Mongiovi, Loris D’antoni, and Bjoern Hartmann. Tracediff: Debugging unexpected code behavior using trace divergences. *2017 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, pp. 107–115, 2017.
- Michele Tufano, Cody Watson, G. Bavota, M. D. Penta, Martin White, and D. Poshyanyk. Learning how to mutate source code from bug-fixes. *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pp. 301–312, 2018.
- Xin Wang, Yasheng Wang, Fei Mi, Pingyi Zhou, Yao Wan, Xiao Liu, Li Li, Hao Wu, Jin Liu, and Xin Jiang. Syncobert: Syntax-guided multi-modal contrastive pre-training for code representation. 2021a.
- Xin Wang, Yasheng Wang, Yao Wan, Jiawei Wang, Pingyi Zhou, Li Li, Hao Wu, and Jin Liu. Code-mvp: Learning to represent source code from multiple views with contrastive pre-training. pp. 1066–1077, 2022.
- Yue Wang, Weishi Wang, Shafiq R. Joty, and S. Hoi. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *ArXiv*, abs/2109.00859, 2021b.
- Chun Xia and Lingming Zhang. Less training, more repairing please: revisiting automated program repair via zero-shot learning. *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022.
- John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. Intercode: Standardizing and benchmarking interactive coding with execution feedback. *ArXiv*, abs/2306.14898, 2023.

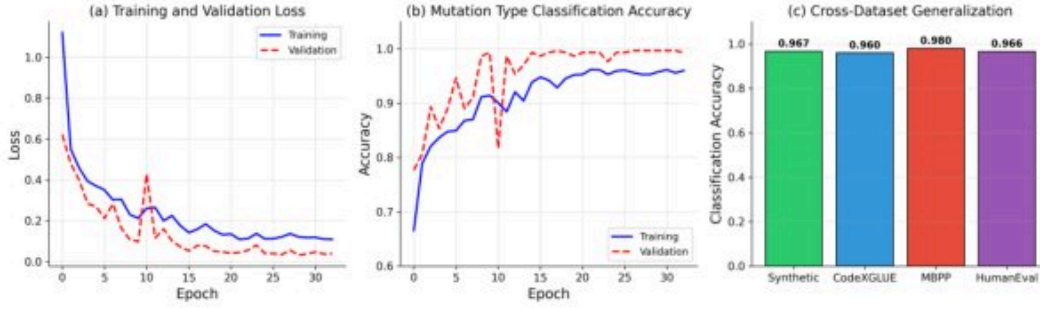


Figure 3: DAPR training dynamics. (a) Loss curves show validation loss below training loss throughout training, an unusual pattern possibly due to the validation set being easier or strong regularization effects. (b) Classification accuracy reaches 96% on training and near-perfect on validation with notable oscillations. (c) Cross-dataset generalization is consistently high (96–98%) across all test distributions.

SUPPLEMENTARY MATERIAL

A TRAINING DYNAMICS

Figure 3 shows complete training dynamics. The unusual pattern of validation loss remaining below training loss throughout training could arise from: (1) the validation set containing “easier” examples with more distinctive divergence patterns, (2) strong regularization (dropout=0.2, weight decay=0.01) affecting training more than validation, or (3) the synthetic data generation process creating systematic differences between splits.

B PER-STEP FEATURE ABLATION DETAILS

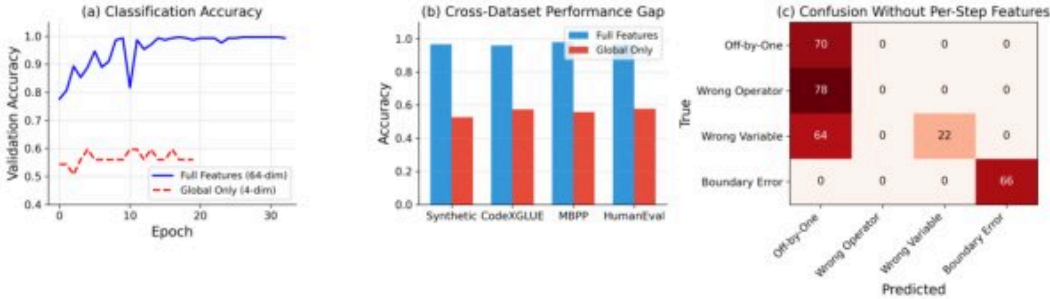


Figure 4: Detailed analysis of per-step trace feature importance. (a) Without per-step features, validation accuracy plateaus at approximately 55%. (b) The performance gap is consistent across all test distributions. (c) Confusion matrix without per-step features reveals catastrophic failure: “wrong operator” is entirely misclassified as “off-by-one” (78/78 instances).

Figure 4 provides detailed analysis of the per-step feature ablation. The confusion matrix reveals that without per-step features, the model cannot distinguish between mutation types that produce similar global divergence patterns.

C OPTIMIZER COMPARISON

The choice of optimizer has dramatic impact (Figure 5). This suggests the loss landscape benefits significantly from adaptive learning rates—the per-step trace features may create parameter interactions that SGD’s uniform learning rate cannot efficiently navigate.

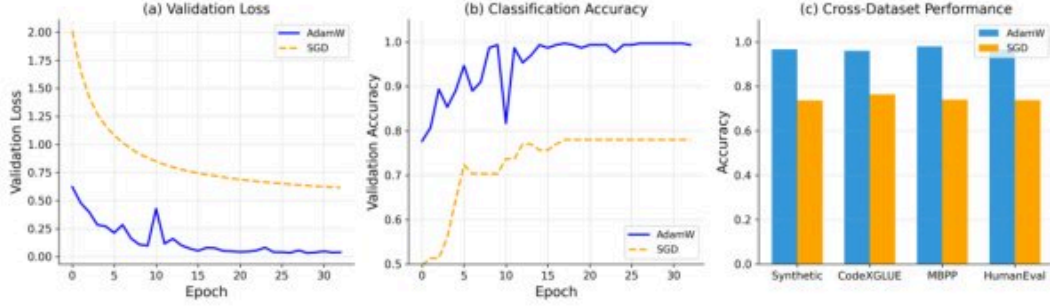


Figure 5: AdamW vs SGD comparison. AdamW achieves approximately 97% accuracy compared to SGD’s approximately 76%, a gap exceeding 20 percentage points consistent across all test datasets.

D ENCODER DEPTH ANALYSIS

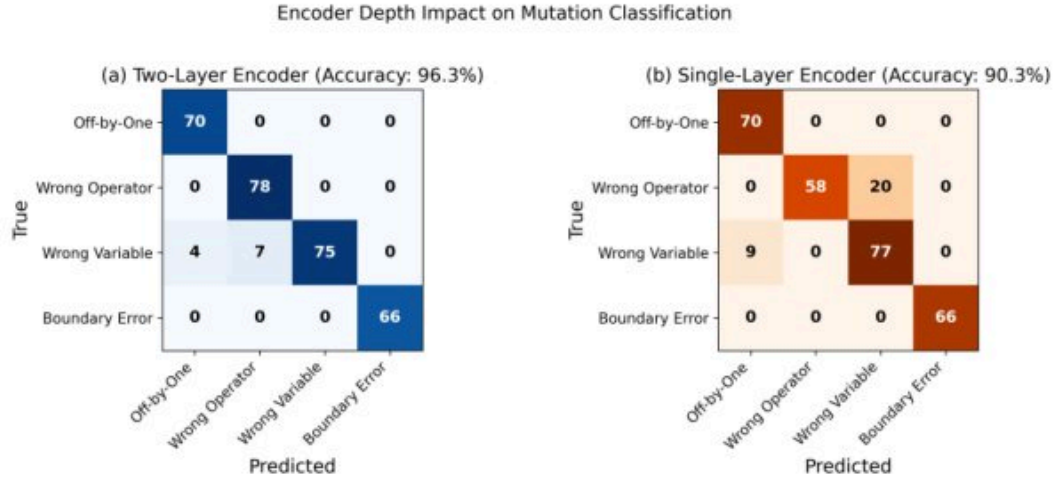


Figure 6: Confusion matrices comparing two-layer encoder (96.3% accuracy) vs single-layer encoder (90.3% accuracy). The single-layer encoder shows substantial confusion: 20 “wrong operator” instances are misclassified as “wrong variable.”

Reducing encoder depth from two layers to one decreases accuracy from 96.3% to 90.3% (Figure 6). The single-layer encoder particularly struggles with “wrong operator” mutations, suggesting that distinguishing operator errors from variable errors requires more representational capacity.

E ATTENTION MECHANISM ANALYSIS

Counter to our hypothesis, attention hurts performance (Figure 7). For fixed-length feature vectors where divergence indicators are already explicitly encoded, attention adds unnecessary complexity without providing useful inductive bias.

F IMPLEMENTATION DETAILS

Program Generation. We generate four program templates: (1) sum computation, (2) factorial, (3) conditional counting, and (4) cumulative sum with lag. Each template has specific mutations for each bug type.

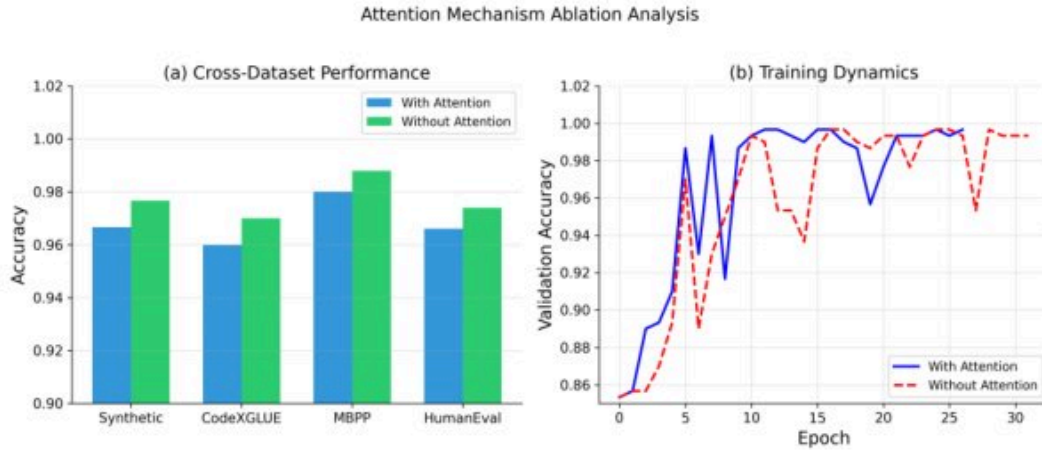


Figure 7: Attention ablation analysis. (a) Removing attention improves accuracy by approximately 1% across all datasets. (b) The attention variant exhibits training instability with pronounced accuracy dips, while the model without attention shows smoother convergence.

Trace Instrumentation. We instrument Python code by inserting trace collection after each assignment, recording variable name-value pairs. Execution is limited to 50 steps to handle potential infinite loops.

Model Architecture. The encoder consists of: $\text{Linear}(64, 128) \rightarrow \text{LayerNorm} \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.2) \rightarrow \text{Linear}(128, 128) \rightarrow \text{LayerNorm} \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.2)$. The classifier head is $\text{Linear}(128, 4)$ with cross-entropy loss.

Hyperparameters. AdamW optimizer with learning rate 0.002, weight decay 0.01, batch size 64, up to 80 epochs with early stopping (patience=15).